



COLEGIO CAPOUILLIEZ

TECNOLOGÍA

INVESTIGACIÓN FORMAL DEL PROYECTO

SMART RIDE GUARD

Sistema de alerta para una zona de seguridad

Integrantes:

Maria Jimena Castro Oliva

Santiago Salazar

Esteban Ramos

Docente: Eswin Godoy

Grado y sección: 11 B

Guatemala, 2026

ÍNDICE

ÍNDICE

• Introducción	1
• Objetivos	4
• Justificación	7
• Marco teórico	9
• Conclusiones	19
• Referencias bibliográficas	21

INTRODUCCIÓN

INTRODUCCIÓN

En lugares con mucha gente, como el IRTRA Petapa, pueden ocurrir situaciones donde alguien se acerque demasiado a un área de peligro. Por eso creamos Smart Ride Guard, un prototipo que detecta cuando una persona u objeto esté a menos de 50 centímetros.

Para hacerlo usamos un Arduino Uno, sensor HC-SR04, LEDs, buzzer, botón, resistencias y protoboard, todo realizado en Tinkercad.

OBJETIVOS

OBJETIVOS

Objetivo general

Diseñar y simular un prototipo electrónico que detecte cuando una persona u objeto se acerca a menos de 50 cm de una zona determinada y que genere una alerta visual y sonora.

Objetivos específicos

- Utilizar un Arduino Uno para procesar la información recibida del sensor ultrasónico.
- Programar el circuito en C++ dentro de Tinkercad.
- Utilizar el sensor HC-SR04 para medir la distancia.
- Mostrar mediante un LED verde o rojo si la zona está segura o invadida.
- Activar un buzzer cuando se detecte una distancia menor de 50 cm.
- Utilizar un botón para silenciar la alarma sonora.
- Relacionar el proyecto con los ODS 9 y 11.

JUSTIFICACIÓN

JUSTIFICACIÓN

Elegimos esta idea porque queríamos crear algo sencillo pero que tuviera una utilidad en un lugar real que podría servir como apoyo para advertir cuando alguien se acerca demasiado a una zona. Su objetivo es demostrar que, con componentes básicos y programación, una idea puede convertirse en un circuito funcional. La propia guía permite plantear una problemática real o inventada y pide explicar cómo el circuito le da una solución.

MARCO TEÓRICO

1. CONTEXTO Y PROBLEMÁTICA DEL PROYECTO

1.1. IRTRA Petapa como contexto del proyecto

El proyecto se desarrolla tomando el Parque Mundo Petapa del IRTRA. La institución señala que está comprometida con brindar un ambiente seguro y con tomar medidas de higiene y seguridad para sus visitantes. Por eso, la seguridad es un tema que tiene sentido dentro del contexto elegido para nuestro prototipo.

(IRTRA, s. f.).

1.2. Problemática planteada

La situación que planteamos consiste en que una persona pueda acercarse demasiado a una zona que debería mantenerse libre. En un lugar con muchas personas, alguien podría no darse cuenta de que está entrando en una zona determinada. Nuestro proyecto busca detectar ese acercamiento de forma automática.

1.3. Propuesta de solución

La solución que propusimos es Smart Ride Guard. El sistema utiliza un sensor ultrasónico para medir la distancia y un Arduino para decidir qué respuesta debe producirse. Cuando la distancia es menor de 50 cm, se enciende el LED rojo y suena el buzzer. Cuando la distancia no está dentro de la zona de alerta, se enciende el LED verde.

1.4. Propósito del prototipo

El propósito principal es demostrar que una idea relacionada con la seguridad puede convertirse en un sistema electrónico funcional. El prototipo también nos permite aplicar programación, conexiones eléctricas y uso de sensores en una situación concreta.

2. COMPONENTES UTILIZADOS

2.1. Arduino UNO

Arduino Uno es la parte principal del circuito porque es el encargado de ejecutar el código y tomar decisiones a partir de la información del sensor. La placa Arduino Uno R3 está basada en el microcontrolador ATmega328P y cuenta con entradas y salidas digitales y entradas analógicas. En nuestro circuito usamos pines específicos para el sensor, los LEDs, el buzzer y el botón. (Arduino, s. f.).

2.2. Sensor ultrasónico HC-SR04

El HC-SR04 es el componente que utilizamos para conocer la distancia. El sensor trabaja con dos señales principales: Trigger, que inicia la medición, y Echo, que permite conocer cuánto tiempo tardó en regresar la señal. En nuestro código, Trigger está conectado al pin A2 y Echo al pin A3. La distancia se calcula usando el tiempo medido por el Echo.

2.3. LEDs y Resistencias

El circuito utiliza dos LEDs para mostrar visualmente el estado del sistema. El LED rojo está conectado al pin 7 y representa una zona invadida. El LED verde está conectado al pin 8 y representa una zona segura. Las resistencias se utilizan junto con los LEDs dentro del circuito para limitar la corriente.

2.4. Buzzer, botón y protoboard

El buzzer está conectado al pin 12 y se utiliza para producir la alerta sonora. El botón está conectado al pin A1 y permite silenciar la alarma. La protoboard facilita la conexión de todos los componentes sin necesidad de soldarlos.

3. FUNCIONAMIENTO DEL CIRCUITO

3.1. Conexiones principales

Las conexiones utilizadas en nuestro código son las siguientes:

Componente	Pin en el código
Trigger del HC-SR04	A2
Echo del HC-SR04	A3
LED rojo	7
LED verde	8
Buzzer	12
Push button	A1

3.2. Medición de distancia

La función `medirDistancia()` comienza poniendo el Trigger en LOW, después genera un pulso HIGH durante 10 microsegundos y finalmente vuelve a LOW. Luego el programa espera la señal del Echo mediante `pulseIn()`. El tiempo recibido se convierte a centímetros utilizando la fórmula incluida en el código: $\text{distancia} = \text{duración} \times 0.034 / 2$.

3.3. Límite de 50 centímetros

El programa tiene una constante llamada DISTANCIA_LIMITE que su valor es 50.

Por eso, cuando la lectura válida es menor que 50 cm, la variable zona Invasada pasa a ser verdadera. Si la lectura es de 50 cm o más, el sistema considera que la zona no está invadida.

3.4. Respuesta del sistema

Cuando zona Invasada es verdadera, el LED rojo se enciende y el LED verde se apaga. Si la alarma no ha sido silenciada, el buzzer produce un tono de 1000 Hz.

4. PROGRAMACIÓN DEL SISTEMA

4.1. Configuración inicial

En `setup()`, el programa establece qué pines serán entradas o salidas. El Trigger y los LEDs, el buzzer y el botón se configuran según la función que cumplen. También se inicia la comunicación serial a 9600 baudios y se enciende el LED verde para comenzar en un estado seguro.

4.2. Función `medirDistancia()`

La función `medirDistancia()` se encarga de realizar una medición. Si `pulseIn()` no recibe un eco dentro del tiempo máximo establecido, la función devuelve -1. Si sí recibe un eco, calcula y devuelve la distancia en centímetros.

4.3. Función `loop()`

En `loop()`, Arduino realiza continuamente una medición y también revisa si el botón está presionado. Si la distancia es válida, actualiza el estado de `zonaInvasada` y muestra la distancia en el monitor serial. Después decide si debe activar el estado seguro o el estado de alarma.

4.4 Silenciar alarma y manejar lecturas invalidas

Cuando la zona está invadida, si se presiona el botón, `alarmaSilenciada` pasa a ser verdadera y el buzzer deja de sonar. Sin embargo, las luces continúan indicando que la zona está invadida. Además, cuando la medición no es válida, el programa no cambia `zonaInvasada` y conserva el último estado válido.

5. ODS, PRUEBAS Y ALCANCE DEL PROTOTIPO

5.1. Relación con el ODS 9

El proyecto se relaciona con el ODS 9, que incluye la innovación y el desarrollo de infraestructura y capacidades tecnológicas. En nuestro caso, utilizamos un sistema electrónico y programación para plantear una solución a una situación relacionada con la seguridad. (Naciones Unidas, s. f.).

5.2. Relación con el ODS 11

También existe una relación con el ODS 11, especialmente con la idea de contar con espacios públicos seguros, inclusivos y accesibles. Nuestro proyecto no pretende resolver por sí solo la seguridad de un parque, pero sí propone un prototipo tecnológico que podría servir como apoyo para detectar el ingreso a una zona determinada. (Naciones Unidas, s. f.).

5.3. Pruebas en Tinkercad

El circuito fue trabajado en Tinkercad Esta simulación permitió revisar las conexiones y comprobar la respuesta del programa. La guía del proyecto establece que el circuito debe estar funcional y probado en Tinkercad y que todos los integrantes deben tener acceso y permisos de edición. (Guía del

Proyecto de Undécimo, 2026).

5.4. Alcance y limitaciones

Smart Ride Guard es un prototipo escolar. Su funcionamiento dentro de Tinkercad no significa que esté listo para instalarse en IRTRA Petapa. Un sistema real necesitaría pruebas adicionales,

componentes adecuados para un ambiente de uso público, criterios de seguridad y una evaluación profesional. Nuestro objetivo en este proyecto es demostrar el funcionamiento de la idea y aplicar lo aprendido en tecnología y programación.

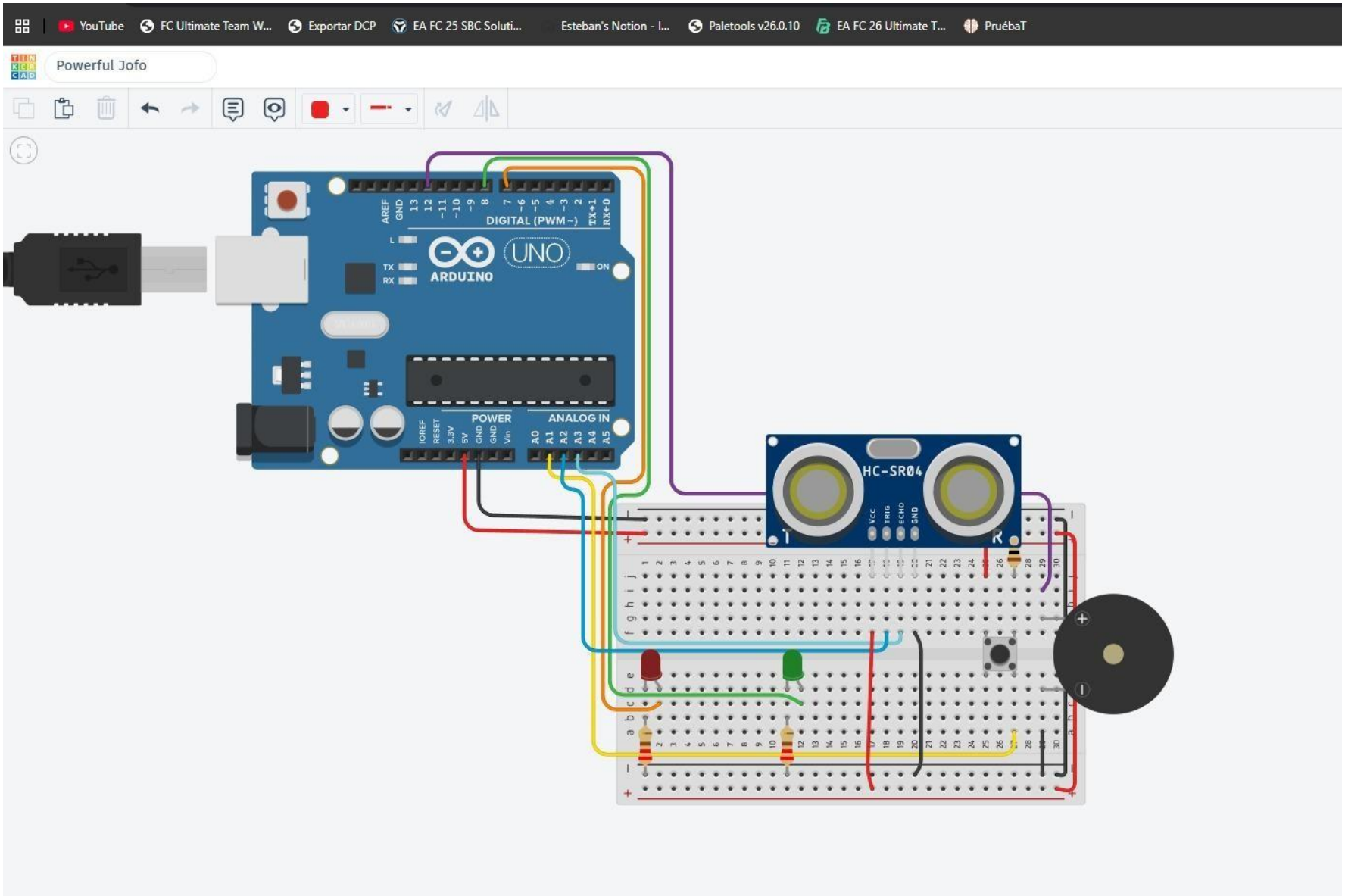


Tabla de funcionamiento

Situación detectada	LED	Buzzer
Distancia menor de 50 cm	Rojo encendido / Verde apagado	Suena, salvo que se silencie con el botón
Distancia de 50 cm o más	Verde encendido / Rojo apagado	Apagado
Lectura inválida	Se conserva el último estado	Se conserva el último estado lógico

CONCLUSIONES

CONCLUSIONES

Con este proyecto logramos convertir una idea de seguridad en un prototipo que realmente funciona y puede detectar distancias en poco tiempo.

- Aprendimos cómo trabajar con Arduino, el sensor y los demás componentes, además de entender mejor cómo funciona la programación en C++.
- Smart Ride Guard nos ayudó a ver cómo la tecnología puede utilizarse para prevenir accidentes y mejorar la seguridad en lugares donde hay muchas personas.

REFERENCIAS

REFERENCIAS BIBLIOGRÁFICAS

Arduino. (s. f.). Arduino UNO Rev3. Arduino Documentation. <https://docs.arduino.cc/hardware/uno-rev3>

Instituto de Recreación de los Trabajadores de la Empresa Privada de Guatemala [IRTRA]. (s. f.). Protocolos de seguridad. <https://irtra.org.gt/protocolos/>

Naciones Unidas. (s. f.). Objetivo 9: Industria, innovación e infraestructura. Departamento de Asuntos Económicos y Sociales. <https://sdgs.un.org/goals/goal9>

Naciones Unidas. (s. f.). Objetivo 11: Ciudades y comunidades sostenibles. Departamento de Asuntos Económicos y Sociales. <https://sdgs.un.org/goals/goal11>

CÓDIGO UTILIZADO

```
// ===== Pines - AJUSTA estos números si no coinciden con tu circuito ===== const int TRIG_PIN = A2; //  
cable verde const int ECHO_PIN = A3; // cable morado const int LED_ROJO = 7; // cable amarillo (alerta)  
const int LED_VERDE = 8; // cable celeste (zona segura) const int BUZZER_PIN = 12; // cable naranja  
const int BUTTON_PIN = A1; // botón
```

```
const int DISTANCIA_LIMITE = 50; // cm - distancia mínima permitida
```

```
bool alarmaSilenciada = false;  
bool zonaInvasada = false; // guarda el último estado válido conocido
```

```
void setup() { pinMode(TRIG_PIN,  
OUTPUT); pinMode(ECHO_PIN, INPUT);  
pinMode(LED_ROJO, OUTPUT);  
pinMode(LED_VERDE, OUTPUT);
```

```

pinMode(BUZZER_PIN, OUTPUT);
pinMode(BUTTON_PIN, INPUT);
Serial.begin(9600);

digitalWrite(LED_VERDE, HIGH); // arranca en estado seguro
}

long medirDistancia() {
  digitalWrite(TRIG_PIN, LOW); delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH); delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  long duracion = pulseIn(ECHO_PIN, HIGH, 25000); // timeout de 25ms (~4m) if (duracion == 0) return -1; //
  lectura inválida, no hubo eco a tiempo

  long distancia = duracion * 0.034 / 2; // conversión a cm return distancia;
}

void loop() {
  long distancia = medirDistancia();
  bool botonPresionado = digitalRead(BUTTON_PIN) == HIGH;

  // Solo actualizamos el estado si la lectura es válida (evita el parpadeo) if (distancia > 0) { zonaInvadida =
  (distancia < DISTANCIA_LIMITE);

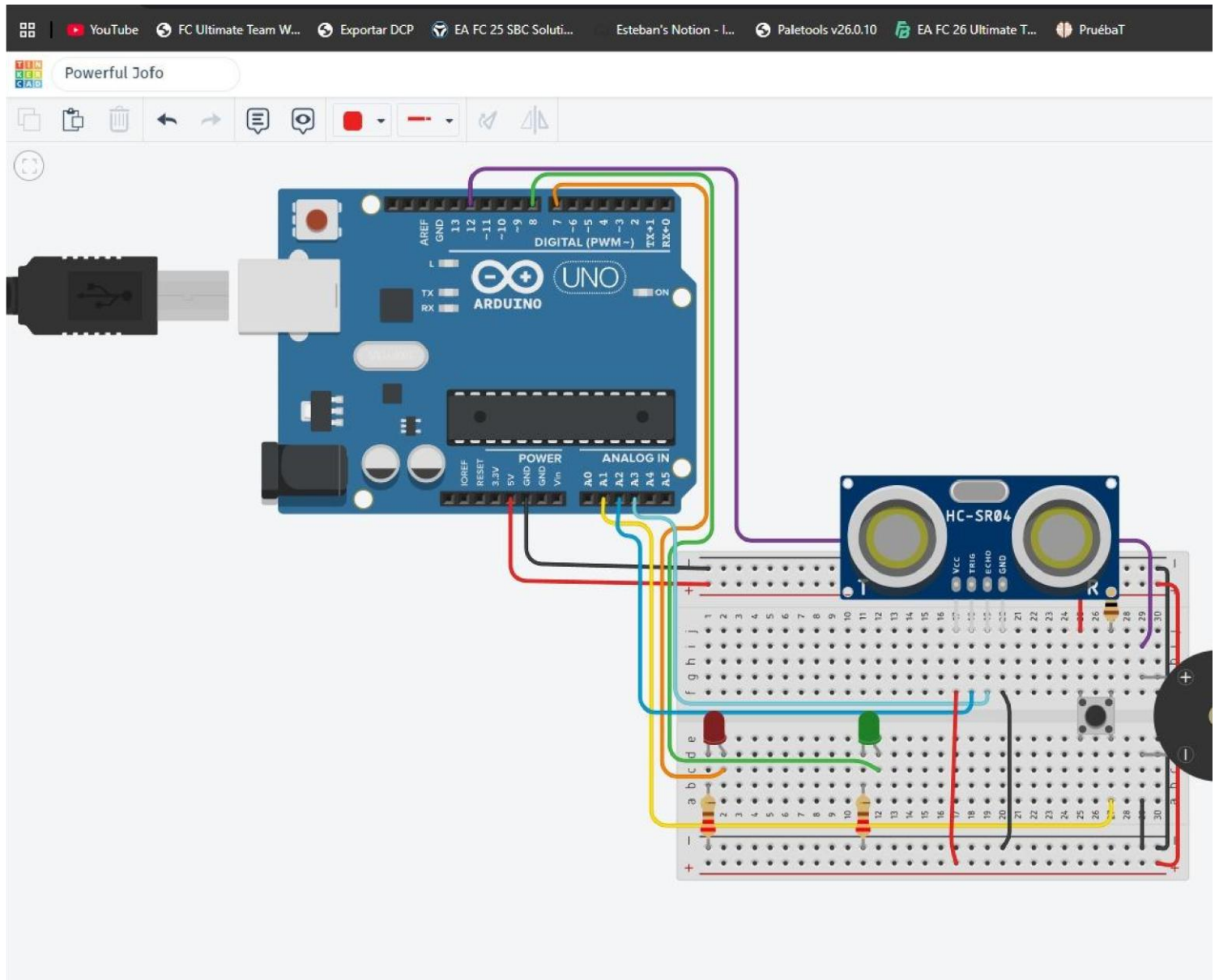
  Serial.print("Distancia: ");
  Serial.print(distancia);
  Serial.println(" cm");
} else {
  Serial.println("Lectura invalida, se mantiene el ultimo estado"); }

  if (zonaInvadida) {
    // Alguien está a menos de 50 cm digitalWrite(LED_ROJO, HIGH);
    digitalWrite(LED_VERDE, LOW);

    if (botonPresionado) { alarmaSilenciada = true;
    }
  }
}

```

```
    if (alarmaSilenciada) {  
noTone(BUZZER_PIN);    } else {  
    tone(BUZZER_PIN, 1000);  
    }  
  
    } else {    // Zona segura  
digitalWrite(LED_ROJO, LOW);  
digitalWrite(LED_VERDE, HIGH);  
noTone(BUZZER_PIN);    alarmaSilenciada =  
false;  
    }  
  
    delay(100);  
}
```



CAPTURA DEL CIRCUITO EN TINKERCAD

